

Credit-Aware Counterfactual Rollout Allocation for LLM Reasoning

Mid-Submission Progress Report — ANLP

Mani Tej Sriram Adithya Casichetty
Addepalli Likhith Bhogadi Sanyam Agrawal
International Institute of Information Technology, Hyderabad
{manitej.sriram, sanyam.agrawal}@research.iiit.ac.in

October 2, 2026

Abstract

Reinforcement learning (RL) for large language model (LLM) reasoning commonly provides feedback only at the end of a reasoning trajectory, making it difficult to determine which intermediate decisions caused the final outcome. Tree-based approaches address this problem by sharing intermediate reasoning states across multiple trajectories, but when the number of rollouts is fixed, where should additional trajectories be generated to obtain useful counterfactual evidence? We propose a two-stage adaptive rollout framework that separates exploration from exploitation: Phase I uses $n - k$ trajectories to construct an exploratory tree using surprisal; Phase II filters candidate states with a perplexity-based novelty proxy and allocates the remaining k rollouts according to observed consequentiality.

In this mid-submission report, we report the complete implementation and empirical findings for **Phase I and the RL baselines**: (1) We ported TreeRL’s training and tree-search infrastructure to run stably on a single 46 GB GPU using DeepSpeed ZeRO-2 and in-memory vLLM colocation. (2) On a rigorous 500-problem Omni-MATH study across 111 configurations, we replicate that tree-based sampling yields 36–41% token savings over i.i.d. chains at matched PassRate. However, we show that *entropy-guided forking does not outperform random forking* (+0.4 to −0.8, all CIs spanning zero). (3) Crucially, our analysis of 168k forks reveals that *surprisal is virtually flat with respect to outcome flips* (5.6–7.5%), while tree position strongly predicts consequence (falling $3\times$ from early to late tokens). This directly justifies our proposed Phase II. (4) On 150-step RL runs evaluated across 1,082 problems, TreeRL outperforms ChainRL by +1.09 points [+0.32, +1.84] pooled, matching the published paper’s dynamic. Our code, logs, and artifacts are available at <https://github.com/likhitbhogadi/TreeRL>.

1 Introduction

Reinforcement learning (RL) has become an effective approach for improving the reasoning capabilities of large language models (LLMs). However, reasoning tasks often provide feedback only through a terminal reward, such as whether the final answer is correct. This creates a fundamental *credit assignment problem*: a long reasoning trajectory may contain many intermediate decisions, while the final reward does not indicate which decisions were responsible for the outcome.

A natural way to obtain more localized credit is to generate multiple trajectories that share intermediate reasoning states. When trajectories diverge from the same state and receive different terminal rewards, their outcomes provide counterfactual evidence about the consequences of decisions at that state. Recent methods exploit this structure for credit assignment and exploration, including TEMPO (Tran et al., 2025), TreePO (Li et al., 2025), and TreeRL (Hou et al., 2025). Other works investigate adaptive rollout allocation (Zhang et al., 2026; Zou et al., 2026) and the use of counterfactual reasoning paths for reducing credit-assignment variance (Ding et al., 2026).

However, generating trajectories can be computationally expensive, making the rollout budget a limited resource. This motivates the following question:

Given a fixed rollout budget, where should additional trajectories be generated to obtain the most useful counterfactual evidence for credit assignment?

Generating all trajectories according to a single exploration criterion may spend computation on states that are uncertain but do not benefit substantially from additional sampling. Conversely, immediately allocating rollouts according to observed rewards may overexploit states before sufficient alternative outcomes have been explored.

We therefore propose a two-stage strategy that explicitly separates *exploration* from *exploitation*. The first stage uses $n - k$ trajectories to construct an ex-

ploratory reasoning tree using surprisal, following the uncertainty-driven expansion principle of TreeRL (Hou et al., 2025). The second stage uses the resulting tree to identify states where additional sampling may produce novel continuations and then allocates the remaining k trajectories according to observed consequentiality. Rather than combining novelty and credit into a single numerical score, novelty is first used to filter the candidate states, after which credit determines how the reserved budget is distributed among the remaining candidates.

2 Background

2.1 Tree-Based Reasoning and Credit Assignment

Tree-based rollout methods represent multiple reasoning trajectories as a shared tree. Each internal node corresponds to an intermediate reasoning state, while different children represent alternative continuations from that state. This structure makes it possible to compare trajectories that share a common reasoning prefix.

Consider a reasoning trajectory

$$\tau = (v_1, \dots, v_m), \quad (1)$$

where each v_i represents a reasoning segment and the trajectory receives a terminal reward $R(\tau)$. For a node v , let $\mathcal{L}(v)$ denote its descendant leaf trajectories. A simple empirical value estimate is

$$V(v) = \frac{1}{|\mathcal{L}(v)|} \sum_{\ell \in \mathcal{L}(v)} R(\ell). \quad (2)$$

When multiple trajectories share v , this value summarizes the observed outcomes associated with that reasoning state. The same tree structure can therefore be used both to obtain localized credit signals and to determine where additional rollout computation should be spent.

3 Related Work

Tree-based reasoning and credit assignment. TEMPO (Tran et al., 2025) exploits shared tree structure to estimate intermediate values from descendant outcomes and obtain more localized credit signals. TreePO (Li et al., 2025) uses segment-level tree rollouts for reasoning optimization, while TreeRL (Hou et al., 2025) performs on-policy tree search by expanding uncertain intermediate states. Our method similarly uses a shared reasoning tree, but explicitly divides the rollout budget between an exploratory stage and a subsequent targeted expansion stage.

Adaptive rollout allocation. IGRPO (Zhang et al., 2026) investigates information-gain-based adaptive rollout allocation in a tree-structured rollout framework,

while TRACE (Zou et al., 2026) studies adaptive allocation of a fixed rollout budget. These works motivate the idea that rollout computation should be allocated selectively rather than uniformly. Our approach similarly treats rollouts as a limited resource, but uses a two-stage procedure in which the first stage builds an exploratory tree and the second stage uses novelty and observed consequentiality to allocate additional trajectories.

Counterfactual reasoning paths. The work by Ding et al. (2026) investigates how alternative reasoning paths can provide useful information for reducing variance in credit assignment. Our approach addresses the complementary question of where such additional counterfactual paths should be generated when only a fixed number of rollouts is available.

4 Proposed Method

4.1 Problem Formulation

For each problem x , let n denote the total rollout budget. We divide it into two portions:

$$n = (n - k) + k. \quad (3)$$

The first $n - k$ trajectories are used for *exploration*, while the remaining k trajectories are reserved for *credit-guided counterfactual expansion*. Each tree node represents a reasoning segment rather than an individual token. Thus, a trajectory forms a path in the tree, and trajectories sharing a prefix reuse the corresponding nodes.

4.2 Phase I: Entropy-Guided Exploration

The first phase constructs an initial tree T_0 using $n - k$ trajectories. The goal of this phase is exploration rather than exploitation. Terminal rewards are therefore not used to determine where trajectories are generated.

Instead, we follow the uncertainty-driven expansion principle of TreeRL (Hou et al., 2025), using local token-level surprisal to identify reasoning states with high uncertainty.

Initial trajectory. We first generate a complete reasoning trajectory from the original problem x . Every reasoning segment becomes a node in T_0 , so the initial tree consists of a single path from the root to a terminal leaf.

Surprisal. Let c_v denote the model context at reasoning state v , and let a be the selected next token. The model assigns a probability $p_\theta(a \mid c_v)$. We compute the surprisal at v as

$$H(v) = -\log p_\theta(a \mid c_v). \quad (4)$$

High surprisal indicates low model confidence in the specific next token. At each rollout step, we select an eligible state with high surprisal and generate a complete continuation from that state. This process continues until exactly $n - k$ trajectories have been generated.

4.3 Phase II: Novelty-Aware Credit-Guided Expansion

After the exploratory tree has been constructed, terminal rewards become available. The remaining k rollouts are reserved for targeted counterfactual expansion.

Perplexity-based novelty. Suppose node v has already produced observed continuations $\mathcal{L}(v)$. For each continuation ℓ , let $S_{\ell,v}$ denote the suffix following v . If this suffix contains $L_{\ell,v}$ tokens, its average negative log-likelihood is

$$H_{\ell,v} = -\frac{1}{L_{\ell,v}} \sum_{t=1}^{L_{\ell,v}} \log p_{\theta}(x_t \mid c_v, x_{<t}). \quad (5)$$

The continuation perplexity is $\text{PPL}_{\ell,v} = \exp(H_{\ell,v})$. For each node with multiple observed continuations, we compute the mean continuation perplexity:

$$D(v) = \frac{1}{|\mathcal{L}(v)|} \sum_{\ell \in \mathcal{L}(v)} \text{PPL}_{\ell,v}. \quad (6)$$

Novelty-based candidate filtering. We retain the top fraction $\rho \in (0, 1]$ of states according to their novelty scores:

$$\mathcal{V}_{\text{novel}} = \text{Top}_{\rho}(\{D(v) : v \in \mathcal{V}(T_0)\}). \quad (7)$$

Local consequentiality. For each candidate node v , we measure the change in estimated value across the decision leading to that node:

$$\delta(v) = V(v) - V(\text{parent}(v)). \quad (8)$$

The consequentiality score is $C(v) = |\delta(v)|$.

Credit-guided allocation. Candidate nodes in $\mathcal{V}_{\text{novel}}$ are converted into an allocation distribution:

$$p(v_i) = \frac{\exp(C(v_i)/\tau)}{\sum_{v_j \in \mathcal{V}_{\text{novel}}} \exp(C(v_j)/\tau)}, \quad (9)$$

where τ is a temperature parameter. For each of the remaining k rollouts, a node v is sampled from $p(v)$, and an independent continuation is generated from $\text{parent}(v)$.

Algorithm 1 Novelty-Aware Credit-Guided Rollout Allocation

Require: Input x , budget n , reserved budget k , temperature τ , candidate fraction ρ

- 1: Initialize tree $T_0 \leftarrow \emptyset$
- 2: Generate complete trajectory ℓ_1 from x ; add to T_0
- 3: **for** tree $i = 1, \dots, M$ **do**
- 4: **for** iteration $l = 1, \dots, L$ **do**
- 5: Compute surprisal $H(v)$ for eligible nodes
- 6: Select top- N high-surprisal branching nodes
- 7: **foreach** selected node v **do**
- 8: Generate T complete continuations from v
- 9: Add trajectories to T_0 ; update tree
- 10: **end for**
- 11: **end for**
- 12: **end for**
- 13: **foreach** eligible node $v \in \mathcal{V}(T_0)$ **do**
- 14: Compute novelty score $D(v)$ via continuation perplexity
- 15: **end for**
- 16: Construct $\mathcal{V}_{\text{novel}} = \text{Top}_{\rho}(\{D(v)\})$
- 17: **foreach** $v \in \mathcal{V}_{\text{novel}}$ **do**
- 18: Compute $V(v)$, $\delta(v) = V(v) - V(\text{parent}(v))$, $C(v) = |\delta(v)|$
- 19: **end for**
- 20: Construct allocation distribution $p(v_i) \propto \exp(C(v_i)/\tau)$
- 21: **for** $j = 1, \dots, k$ **do**
- 22: Sample $v \sim p(v)$
- 23: Generate continuation from $\text{parent}(v)$; add to T_0
- 24: Update relevant tree statistics
- 25: **end for**
- 26: Recompute tree values and credit-derived advantages
- 27: Perform policy optimization step
- 28: **return** Updated policy and completed tree

Figure 1: The proposed rollout allocation framework.

4.4 Final Credit Assignment and Policy Optimization

After counterfactual expansion, the complete tree is used to derive variable, node-level credit estimates for policy optimization. Intermediate reasoning states receive credit signals reflecting their observed position and counterfactual outcomes.

4.5 Algorithm

Algorithm 1 outlines the full procedure.

5 Dataset and Experimental Setup

5.1 Datasets

Training problems. For RL we use `train_30k`, the 30,000-problem file that Hou et al. (2025) release with their training script. Each problem comes with a final answer and no reference solution. By source tag, it contains WebInstruct (26%), NuminaMath (24%) and its

cn_k12 subset (17%), Omni-MATH (13%), and smaller sources.

A tree whose leaves are all correct or all wrong yields zero advantage under tree-based credit, so it contributes no gradient. We therefore filter by difficulty. We sample 3,000 problems and let the base model answer each 8 times (temperature 1.0): 42% are never solved and 10% always. We keep the 1,432 problems solved 1–7 times out of 8. Our long RL runs use 1,200 of them ($150 \text{ steps} \times 8 \text{ problems}$), each seen once.

Inference-only study (Phase I). The EPTree study in Section 6.1 needs no training data. It uses 500 Omni-MATH problems (Gao et al., 2024), a random sample (seed 0) without any filtering, which is the same construction as the paper’s Omni-MATH-500. We chose hard, unfiltered problems because PassRate (at least one correct response) saturates on easier sets: an earlier run on 243 accuracy-filtered OlympiadBench problems reached 95% PassRate with 16 i.i.d. chains, leaving no room to compare samplers. On our Omni-MATH sample the base model solves 26% of problems with one response and 53% with 64.

Evaluation benchmarks. We evaluate on three sets, 1,082 problems in total:

- MATH500 (Hendrycks et al., 2021; Lightman et al., 2024): 500 problems.
- AMC: 83 problems from the AMC 12A/12B contests of 2022 and 2023.
- Omni-MATH-500: a random sample (seed 0) of 500 Omni-MATH problems (Gao et al., 2024). We score 499 of them; one prompt exceeds the 1,024-token limit.

Omni-MATH-500 overlaps train_30k: 14 of its problems, and 1 MATH500 problem, are among our 1,200 training problems. We report results with these 15 problems removed as well.

Reward and grading. The reward is binary: 1 if the last `\boxed{}` answer matches the reference under `math-verify`, else 0. A response cut off at the length limit counts as wrong.

5.2 Model and Training

Model. All experiments use Qwen2.5-Math-1.5B-Instruct (Yang et al., 2024) with its default chat template. We sample at temperature 1.0 and top- p 0.95. At the paper’s temperature of 1.2, this model produced degenerate output in 13% of samples. Responses are capped at 3,072 tokens for the Phase I study and 2,048 for RL; only 1.1% of the model’s responses exceed 2,048 tokens.

	TreeRL paper	Ours
Policy	7–14B	Qwen2.5-Math-1.5B
RL steps	~300	40 / 150
Learning rate	1.5e-6	1.5e-6 / 5e-6
Problems / step	16	8
Trained resp./problem	30 Tree, 16 Chain	8
KL β	10^{-4}	0
Temperature, top- p	1.2, 0.95	1.0, 0.95
Max response tokens	8,192	2,048
Optimiser	ZeRO-3, CPU Adam	ZeRO-2, CPU Adam

Table 1: RL settings: the TreeRL paper and ours. Each of our steps uses 64 trained responses and one optimiser update.

Training stack. We reuse the TreeRL authors’ released implementation unchanged for the learning algorithm itself: EPTree tree construction, tree-derived advantages and the policy-gradient loss. We port its infrastructure to a single 46 GB GPU:

- the policy (DeepSpeed ZeRO-2, CPU Adam) and the vLLM sampler share the GPU;
- updated weights reach vLLM through CUDA IPC instead of NCCL;
- the trees of all problems in a step are sampled in one batched vLLM call.

Peak GPU memory is 18.5–21 GB, so 1.5B-parameter RL fits comfortably in a 40 GB budget.

Methods compared. All three methods share the same code path and differ only in how responses are sampled and credited:

- **TreeRL:** EPTree with $(M, N, L, T) = (6, 2, 1, 2)$, i.e. 30 leaves per problem. 8 leaves are trained on, with TreeRL’s tree advantages (global plus local value differences, scaled by $1/\sqrt{|\mathcal{L}(v)|}$).
- **ChainRL:** 8 independent chains through the same code. Without branching, the tree advantage reduces to a leave-one-out (RLOO) baseline.
- **GRPO** (Shao et al., 2024): the same 8 chains with the group-normalised advantage $(r_i - \bar{r})/\text{std}(r)$. The KL penalty is set to $\beta = 0$, as for the other two methods.

Table 1 lists the settings next to the paper’s.

Evaluation protocol. We report greedy accuracy and mean@8, the accuracy averaged over 8 samples per problem at temperature 1.0. Because scoring the same model twice varies by about 0.5 points, models are compared on the same problems with a paired bootstrap: 2,000 resamples of problems, 95% CI.

6 Results

6.1 Phase I: Surprisal-Guided Exploration

Phase I of our method builds the exploratory tree with EPTree’s surprisal criterion. We first replicate EPTree as an inference-only study on 500 Omni-MATH problems (Section 5.1), using TreeRL’s own tree construction code. We compare three samplers:

- EPTree, which forks at the N highest-surprisal tokens (6 tree shapes);
- random forking with the same (M, N, L, T) , the paper’s ablation (4 of the shapes);
- i.i.d. chains: 64 per problem, from which $\text{pass}@k$ is estimated without bias for any $k \leq 64$.

Table 2 lists all eleven runs.

Trees are a cheaper way to search. Every tree run lies on or above the i.i.d. curve (Figure 2a,b), but the gains are modest. Eight runs can be compared with i.i.d. chains at the same token budget; their gain ranges from +0.4 to +3.3 points and is significant for three of them: EPTree (2, 3, 1, 1) and both (6, 2, 1, 2) runs.

Expressed as cost, those three reach the PassRate of i.i.d. chains with 36–41% fewer tokens. For example, EPTree (6, 2, 1, 2) needs 14.5k tokens per problem where i.i.d. chains need 23.9k: a 40% saving [15%, 57%]. For the other five runs the savings are 7–20% and not significant (Table 2). The two (8, 4, 2, 2) runs exceed the 64-chain budget, so we do not compare them.

The saving does not come from better individual answers: mean per-response accuracy is 25.7–26.2% for every tree run, the same as a single chain (25.9%). It comes from sharing prefixes, which makes each additional response cheaper. Trees also give more distinct final answers per token: 8.4 for (6, 2, 1, 2), against about 7.0 for i.i.d. chains at the same budget.

The gain comes from hard problems. To see which problems benefit, we split them by how many of 32 held-out i.i.d. chains solve them, and compare the trees against the other 32 chains at the same token budget (Figure 2c). On the 256 problems that no held-out chain solves, the (6, 2, 1, 2) EPTree trees gain +3.3 points [+0.4, +6.5]. On problems solved sometimes, the gain is +4.3 [−2.0, +11.1]; with 119 problems this interval is wide. On problems solved by most chains it is 0, since both samplers are at ceiling. Trees help exactly where independent sampling rarely finds a solution.

Trees produce more learnable groups. In RL, a problem yields a gradient only when its responses include both a correct and an incorrect one. At the same token budget, trees produce such mixed groups more often than i.i.d. chains (Table 2, Mixed): 39.4% vs. 34.4%

of problems for (6, 2, 1, 2), and 26.6% vs. 21.4% for (2, 3, 1, 1). This is a plausible mechanism for TreeRL’s advantage over ChainRL in Section 6.2.

Where to fork matters little for PassRate. At identical (M, N, L, T) , entropy-guided and random forking differ by +0.4, +0.8, 0.0 and −0.8 points, and every paired CI includes zero. The paper’s differences (+2.1, +1.0) lie inside our intervals. In this setting the efficiency gain comes from sharing prefixes, not from choosing the fork point.

Two of the paper’s descriptive results replicate. Fork positions are spread across the response (mean relative position 0.50). The most frequent fork tokens are function words and L^AT_EX delimiters (“the”, “\(", “and”); unlike the paper’s model, ours never writes “Wait”.

Surprisal does not predict consequential forks; position does. For credit assignment, the useful forks are those whose branch reaches a different outcome than the parent. Across all 168k forks of the ten tree runs, the share of branches that flip correctness is flat in the surprisal of the token forked at: between 5.6% and 7.5% in every surprisal bin, from below 0.01 nats to above 8 nats. It falls steadily with the fork’s position instead, from 9.7% [9.0, 10.5] for forks in the first fifth of a response to 3.4% [2.9, 3.8] in the last fifth (95% bootstrap CIs over trees).

Comparing the two samplers directly, EPTree branches change the final answer more often than random branches (46.6% vs. 42.6%, paired +4.0 [+3.2, +4.8]). They change correctness only slightly more often (7.0% vs. 6.1%, +0.9 [+0.5, +1.3]). Over 90% of branches from either sampler reproduce their parent’s outcome.

Implications for our method. Surprisal is a reasonable signal for *diversity*: EPTree’s branches give more distinct answers. It is a weak signal for *consequentiality*, the property credit assignment needs. This supports our proposal in three ways:

- It confirms the limitation stated in Section 9: surprisal measures local uncertainty, not downstream usefulness.
- It motivates Phase II. Most rollouts spent on surprisal-chosen states produce no counterfactual evidence, so the reserved k rollouts should go where observed outcomes differ.
- It suggests position, or the remaining length after a state, as a cheap additional feature for the Phase II candidate set, which our ablations will test.

6.2 RL Baselines

Short runs. With 40 steps at the paper’s learning rate, none of the three methods changes the model measurably. On greedy MATH500, for example, step-40

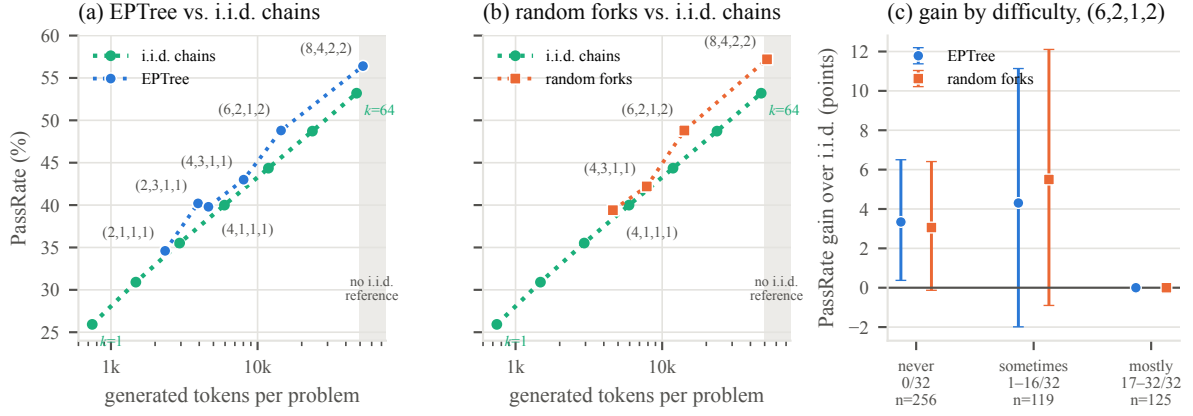


Figure 2: Tree sampling vs. i.i.d. chains on 500 Omni-MATH problems. (a,b) PassRate (at least one correct response) against generated tokens per problem, for EPTree (a) and random forking (b), each against the same i.i.d. curve ($k = 1, 2, \dots, 64$ chains). Each point is one (M, N, L, T) run; dotted lines join runs in order of token cost. A point above the i.i.d. curve reaches that PassRate with fewer tokens. The shaded band lies beyond 64 chains, where no matched i.i.d. comparison exists. (c) Gain of the $(6, 2, 1, 2)$ trees over i.i.d. chains at the same token budget, by problem difficulty, with 95% bootstrap CIs. Difficulty is how many of 32 held-out chains (chains 33–64) solve the problem. The i.i.d. reference uses the other 32 chains, so the two are independent.

checkpoints score 74.4–75.4 against the base model’s 74.6, and every difference across the three benchmarks is within noise. We therefore ran 150 steps with a higher learning rate (5×10^{-6}).

Long runs. TreeRL and ChainRL are indistinguishable at steps 50 and 100 (Figure 3a). By step 150, TreeRL improves over the base model by +0.92 points [+0.17, +1.66], while ChainRL stays at the base level (−0.16 [−0.85, +0.52]).

Head-to-head on the same problems, TreeRL leads ChainRL by +1.09 [+0.32, +1.84] (Figure 3b, Table 3). It is ahead on every benchmark: MATH500 +1.07, AMC +2.41, Omni-MATH-500 +0.88. Excluding the 15 test problems that overlap the training data gives +1.04 [+0.27, +1.79].

The pattern matches the paper, where both methods track each other early and TreeRL pulls ahead after about 100 of ~300 steps. The size of the gap is close to the paper’s for an already strong policy: +1.3 for R1-Distill-Qwen-7B.

Caveats.

- Each method was trained once, so run-to-run variance is not yet measured.
- The methods are matched on trained responses, not on generated tokens. Our TreeRL generates about $2.4\times$ more tokens than ChainRL, whereas the paper matches generated tokens.
- Our budget (150 steps $\times$ 64 responses) is roughly 1/15 of the paper’s. The base policy, already tuned for math by its developers, leaves little headroom on MATH-level problems.

- The GRPO baseline has completed 50 of its 150 steps so far.

6.3 Summary of Findings

- **Tree sampling is somewhat cheaper than i.i.d. sampling.** At the same token budget, tree runs gain +0.4 to +3.3 points PassRate, significantly for 3 of 8 runs; those three need 36–41% fewer tokens for the same PassRate. The gain comes from shared prefixes and concentrates on problems that independent sampling rarely solves. Trees also produce more distinct answers and more problems with mixed outcomes.
- **Fork placement does not matter for PassRate.** Entropy-guided forking does no better than random forking at equal tree shape (all paired CIs include zero).
- **Surprisal predicts diversity, not consequence.** The chance that a branch flips correctness is flat in surprisal but falls about $3\times$ from early to late forks, and over 90% of branches reproduce their parent’s outcome. This directly motivates our Phase II.
- **TreeRL beats ChainRL in RL, modestly.** With identical settings and budgets, TreeRL improves over ChainRL by +1.09 points [+0.32, +1.84] after 150 steps; the two are indistinguishable earlier. The gap is close to the paper’s for an already strong policy.
- **Single-GPU feasibility.** RL on a 1.5B model, with on-policy tree sampling, runs on one 46 GB GPU at a peak of 18.5–21 GB.

Sampler	(M, N, L, T)	Leaves	Tokens	PassRate	Gain [95% CI]	Tokens saved [95% CI]	Distinct	Mixed (%)
i.i.d.	$k = 4$	4	2.9k	35.5	—	—	2.51	18.3
i.i.d.	$k = 8$	8	5.9k	40.0	—	—	3.92	25.8
i.i.d.	$k = 16$	16	11.9k	44.4	—	—	6.00	32.6
i.i.d.	$k = 32$	32	23.7k	48.7	—	—	9.30	38.9
i.i.d.	$k = 64$	64	47.3k	53.2	—	—	14.41	45.0
EPTree	(2,1,1,1)	4	2.3k	34.6	+0.6 [−1.3, +2.6]	9% [−21, +34]	2.32	16.6
EPTree	(2,3,1,1)	8	3.9k	40.2	+2.8 [+0.6, +5.0]	36% [+9, +55]	3.56	26.6
EPTree	(4,1,1,1)	8	4.6k	39.8	+1.4 [−0.6, +3.5]	20% [−11, +43]	3.65	24.6
Random	(4,1,1,1)	8	4.6k	39.4	+1.0 [−1.0, +3.2]	15% [−16, +41]	3.63	23.8
EPTree	(4,3,1,1)	16	8.0k	43.0	+1.1 [−0.9, +3.2]	16% [−16, +42]	5.53	31.6
Random	(4,3,1,1)	16	7.9k	42.2	+0.4 [−1.6, +2.5]	7% [−28, +32]	5.38	29.8
EPTree	(6,2,1,2)	30	14.5k	48.8	+3.2 [+1.0, +5.3]	40% [+15, +57]	8.41	39.4
Random	(6,2,1,2)	30	14.2k	48.8	+3.3 [+1.2, +5.6]	41% [+18, +60]	8.07	38.4
EPTree	(8,4,2,2)	136	52.4k	56.4	—	—	21.07	48.4
Random	(8,4,2,2)	136	51.9k	57.2	—	—	19.92	48.2

Table 2: All Phase I runs on 500 Omni-MATH problems. *Tokens*: generated tokens per problem, counted once per shared prefix. *Gain*: PassRate minus i.i.d. pass@ k at the same token budget (interpolated log-linearly in tokens). *Tokens saved*: how many fewer tokens the tree uses than i.i.d. chains need to reach its PassRate. *Distinct*: distinct final answers per problem. *Mixed*: share of problems with at least one correct and one incorrect response. (8, 4, 2, 2) exceeds the 64-chain budget, so it has no matched i.i.d. comparison. EPTree minus Random at the same shape (paired, PassRate points): +0.4 [−2.0, +2.8], +0.8 [−1.6, +3.4], 0.0 [−2.4, +2.4], −0.8 [−3.2, +1.6]. Mean per-response accuracy is 25.7–26.2% for every tree run, against 25.9% for a single i.i.d. chain.

	MATH500	AMC	Omni	Pooled	Mean
Base	73.95	45.03	26.68	49.93	48.55
ChainRL	73.83	45.78	26.33	49.77	48.65
GRPO	step 50 evaluated (−0.37 vs base)				
TreeRL	74.90	48.19	27.20	50.85	50.10
TreeRL − ChainRL, pooled: +1.09 [+0.32, +1.84]					

Table 3: Mean@8 accuracy (%) at step 150 of the long runs. Pooled: over all 1,082 problems. Mean: un-weighted mean of the three benchmarks, as the paper reports.

7 Future Evaluation Plan

In the second phase of the project, we will implement and evaluate Phase II:

1. **Phase II Implementation:** Implement continuation perplexity calculation $D(v)$, candidate filtering $\mathcal{V}_{\text{novel}}$, and local consequentiality-based credit allocation $\delta(v) = V(v) - V(\text{parent}(v))$.
2. **Counterfactual Allocation Quality:** Evaluate whether states selected for additional expansion exhibit greater downstream outcome variation than surprisal-only or random baselines.
3. **Ablation Suite:**
 - Credit-guided allocation without novelty filtering.
 - Novelty filtering followed by credit-guided allocation.
 - Novelty-only allocation vs. uniform/random allocation.

- Varying candidate fraction ρ and reserved budget ratio k/n .

4. **RL with Phase II Trees:** Train the policy with the completed Phase I + Phase II trees under matched total rollout budgets against TreeRL and ChainRL.

8 Expected Contributions

The project aims to make three contributions:

1. We formulate adaptive rollout allocation as a credit-aware exploration problem, treating additional trajectories as a limited resource for obtaining counterfactual evidence.
2. We propose a two-stage rollout allocation framework that separates uncertainty-driven exploration from novelty-aware, credit-guided exploitation.
3. We provide thorough empirical justification (Phase I replication, surprisal/consequentiality discrepancy analysis, and full RL baselines) showing why local uncertainty is insufficient for counterfactual credit assignment.

9 Limitations and Future Work

The Phase I exploration strategy follows TreeRL’s uncertainty-driven expansion and relies on surprisal as a proxy for exploratory potential. As our mid-submission findings show, surprisal reflects uncertainty over immediate tokens but does not predict whether an alternative branch flips final-answer correctness. Furthermore, continuation perplexity is only a proxy for

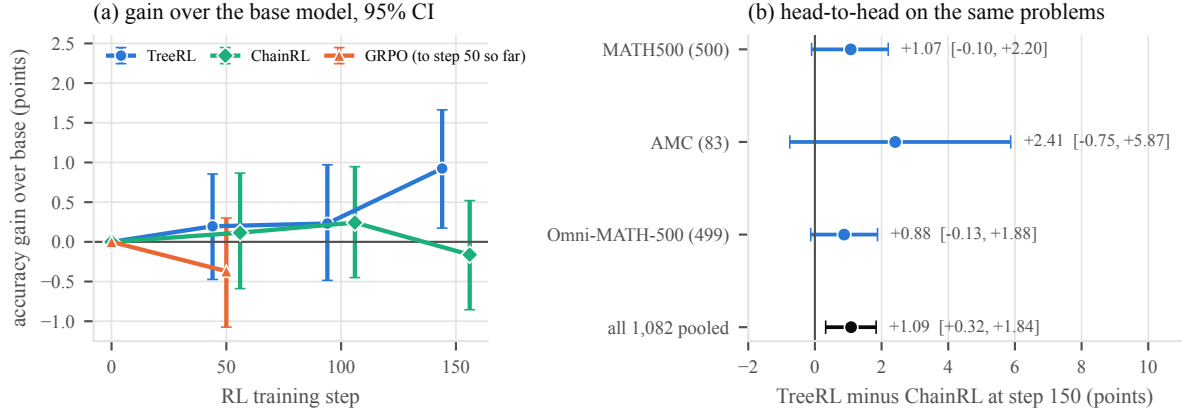


Figure 3: RL with Qwen2.5-Math-1.5B-Instruct: 150 steps, 8 problems $\times$ 8 trained responses per step. Accuracy is mean@8 on the same 1,082 test problems, with paired bootstrap 95% CIs. (a) Gain over the base model during training. GRPO is evaluated up to step 50 so far. (b) TreeRL minus ChainRL at step 150, per benchmark and pooled.

novelty, and local consequentiality $\delta(v)$ is one possible measure of decision importance. Finally, our experiments focus on verifiable mathematical reasoning tasks; extending the framework to general reasoning and interactive environments remains future work.

10 Project Status and Timeline

Completed milestones (Mid-Submission):

- **Week 1 (Oct 2026):** Setup standalone TreeRL harness, single-GPU infrastructure (colocated DeepSpeed ZeRO-2 + vLLM, memory-aware scheduling).
- **Week 2 (Oct 2026):** Full Phase I empirical study on 500 Omni-MATH problems across 11 tree and chain configurations; extensive surprisal and consequentiality analysis (168k forks).
- **Week 2–3 (Oct 2026):** 150-step RL baseline runs (TreeRL, ChainRL, GRPO) on 1,200 difficulty-filtered problems, evaluated on 1,082 problems with paired bootstrap CIs.

Remaining milestones (Final Submission):

- **Week 3 (Oct 2026):** Implement Phase II novelty filtering and consequentiality-based rollout allocation; integrate with tree construction manager.
- **Week 4 (Oct 2026):** Run Phase II counterfactual quality benchmarks, rollout allocation ablations, and policy optimization comparisons; complete final report.

References

- Fei Ding, Yongkang Zhang, Yeling Peng, Youwei Wang, Guoxiong Zhou, and Zijian Zeng. Reducing credit-assignment variance through counterfactual reasoning paths. *arXiv preprint arXiv:2605.16302*, 2026.
- Bofei Gao, Feifan Song, Zhe Yang, Zefan Cai, Yibo Miao, Qingxiu Dong, Lei Li, Chenghao Ma, Liang Chen, Runxin Xu, et al. Omni-MATH: A universal olympiad level mathematic benchmark for large language models. *arXiv preprint arXiv:2410.07985*, 2024.
- Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the MATH dataset. In *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*, 2021.
- Zhenyu Hou, Ziniu Hu, Yujiang Li, Rui Lu, Jie Tang, and Yuxiao Dong. TreeRL: LLM reinforcement learning with on-policy tree search. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL 2025)*, 2025.
- Yizhi Li, Qingshui Gu, Zhoufutu Wen, Ziniu Li, Tian-shun Xing, Shuyue Guo, Tianyu Zheng, Xin Zhou, Xingwei Qu, Wangchunshu Zhou, et al. TreePO: Bridging the gap of policy optimization and efficacy and inference efficiency with heuristic tree-based modeling. *arXiv preprint arXiv:2508.17445*, 2025.
- Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let’s verify step by step. In *International Conference on Learning Representations (ICLR)*, 2024.

Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. DeepSeek-Math: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.

Hieu Tran, Zonghai Yao, and Hong Yu. Exploiting tree structure for credit assignment in reinforcement learning with large language models. *arXiv preprint arXiv:2509.18314*, 2025.

An Yang, Beichen Zhang, Binyuan Hui, Bofei Gao, Bowen Yu, Chengpeng Li, Dayiheng Liu, Jianhong Tu, Jingren Zhou, Junyang Lin, et al. Qwen2.5-Math technical report: Toward mathematical expert model via self-improvement. *arXiv preprint arXiv:2409.12122*, 2024.

Yijun Zhang, Fan Xu, Jiaxin Ding, Yule Xie, Shiqing Gao, Xin Ding, Haoxiang Zhang, Luoyi Fu, and Xinbing Wang. Information gain-based rollout policy optimization: An adaptive tree-structured rollout approach for multi-turn LLM agents. *arXiv preprint arXiv:2607.06223*, 2026.

Heming Zou, Qi Wang, Yun Qu, Yuhang Jiang, Lizhou Cai, Yixiu Mao, Ru Peng, Xin Xu, Weijie Liu, Kai Yang, et al. TRACE: A unified rollout budget allocation framework for efficient agentic reinforcement learning. *arXiv preprint arXiv:2606.11119*, 2026.